Formalisme AAM O94_3 : observation passive en direct, apprentissage de briques et intégration locale

Dossier technique AAM

Version 0.94.3 — 19 octobre 2026

Contents

1 Objet, portée et obligations 1

2 Architecture générale 2

3 Définition normative de l'observation en direct 2

3.1 Niveau filesystem . . . 2
3.2 Niveau processus . . . 2
3.3 Niveau réseau . . . 2
3.4 Protocole actif JSONL distinct . . . 3

4 Trace créée par AAM 3

5 Apprentissage et promotion de briques 3

6 Programmation par composition et compilation 3

7 Modèles locaux et endpoints 4

8 Ergonomie 4

9 Préservation du RAG historique 4

10 Commandes d'exécution 4

11 Protocole de validation et interprétation 5

12 Conséquences 5

13 État et limites de preuve 5

1 Objet, portée et obligations

O94_3 est une incrémentation corrective de O94_2. L'objectif opérationnel est que la commande *Observer* démarre une source vivante sélectionnée, collecte elle-même les événements observables, écrit la trace au fil de l'eau, conserve la provenance, puis rend cette trace utilisable pour analyse et apprentissage de briques. La trace JSONL est un résultat de l'observation, jamais un prérequis du mode passif.

Le système distingue les états `PASS_BOUNDED`, `PARTIAL`, `UNRESOLVED`, `DEPENDENCY_MISSING` et `ERROR`. L'exécution locale, une concordance sur les données vues ou une compilation de graphe ne constituent pas en elles-mêmes une preuve de généralisation. Aucun résultat universel n'est postulé a priori.

2 Architecture générale

Composant	Responsabilité
`PassiveObservationSession`	Session asynchrone bornée, démarrée avec autorisation explicite, qui observe une source sélectionnée et crée sa trace append-only.
`observation_live.py`	Adaptateurs filesystem, PID/processus et interface réseau via `tshark`; budget et rapport d'arrêt.
`AAMRuntime`	Orchestre les sessions, inscrit les événements de cycle de vie dans la mémoire et fournit l'accès au catalogue des briques apprises.
Interface Observer	Choix du niveau, des sources, budgets, sortie automatique de trace, état et arrêt; les onglets denses sont enveloppés dans un canvas avec barre verticale.
`LearnedBlockStore`	Analyse bornée des traces; crée des correspondances exactes à support fini seulement si les entrées/sorties train sont non ambiguës; extrait également des candidats de schéma et de séquence soumis à examen.
`compile_composition`	Compile le DSL déclaratif vers un graphe de compétences/règles connu et écrit un artefact JSON versionné. Aucun code arbitraire n'est lancé par le compilateur de composition.
`IntegratedModelRegistry`	Intégration des Transformers locaux, TorchScript PyTorch tensoriel, adaptateurs PyTorch explicites pour state_dict, endpoints Ollama et API compatibles OpenAI, et autre workspace AAM. La politique d'entraînement est propre au backend.
`SemanticRAG` / moteur O92	Espaces sémantiques, poids, classement, retrieval, mémoire et résolution historiques préservés byte-à-byte dans les modules RAG indiqués au registre d'intégrité.

3 Définition normative de l'observation en direct

3.1 Niveau filesystem

L'utilisateur sélectionne un répertoire concret. Le collecteur observe de façon répétée les fichiers ordinaires ajoutés, modifiés ou supprimés sous ce périmètre, sans suivre les liens symboliques. Il enregistre chemin relatif, taille, horodatage de modification, mode et, si choisi, hash de contenu. Le contenu brut est désactivé par défaut. S'il est activé, la capture est limitée à 65 536 octets par fichier par défaut et à un budget total explicite. Le fichier de trace, son rapport et ses métadonnées sont exclus de l'arbre surveillé afin d'éviter une boucle d'auto-capture.

3.2 Niveau processus

L'utilisateur sélectionne un PID. Lorsque `psutil` est disponible et que les droits système l'autorisent, la session échantillonne nom, statut, création, usage CPU, threads, mémoire résidente, fichiers ouverts et métadonnées des sockets associés au processus. Un accès refusé est rapporté comme tel. Cette voie ne lit pas arbitrairement la mémoire privée du processus, ne reconstitue pas son exécution interne et ne contourne pas les permissions du système.

3.3 Niveau réseau

L'utilisateur sélectionne une interface réseau (répertoriée via `tshark`) et un filtre BPF facultatif. La capture requiert Wireshark/TShark installé ainsi que les droits système/capture adaptés. Par défaut, la trace contient métadonnées de trame, adresses visibles, ports et drapeaux. Les charges utiles sont désactivées par défaut et exigent un opt-in. Les contenus TLS restent chiffrés; aucune décryption ni collecte totale implicite n'est revendiquée.

3.4 Protocole actif JSONL distinct

L'ancien canal de boîte noire qui démarre un processus persistant et lui envoie des valeurs JSON Lines reste disponible comme mode séparé. Il observe les couples entrée/réponse produits par ce protocole. Il ne doit pas être présenté comme observation passive d'une application arbitraire. Le chargement d'un dataset prêt à l'emploi demeure un troisième parcours: import/réexécution, non capture live.

4 Trace créée par AAM

Chaque session crée un nouveau fichier JSONL dans le dossier de captures du workspace ou dans le dossier explicitement choisi. Chaque ligne comprend le schéma, l'ID de session, un numéro de séquence strictement croissant, les horodatages, la source, le type d'événement, les données et une chaîne d'intégrité `prev_hash`/`event_sha256`. Après arrêt, AAM écrit un rapport JSON et des métadonnées latérales contenant les hashes, les budgets utilisés, la cause d'arrêt, le nombre d'événements et les erreurs observées.

Budgets exposés: durée maximale, nombre d'événements, intervalle de polling, budget total en octets, taille maximale d'événement et opt-in payload. Des plafonds durs protègent l'exécution contre une valeur de configuration démesurée. Dépasser un budget produit un état explicite et préserve ce qui a déjà été observé.

5 Apprentissage et promotion de briques

Les traces se distinguent par leur sémantique:

- Une trace instrumentée comportant explicitement `input` et `target`/`output` peut produire une compétence de lookup exact sur les entrées non ambiguës du split train. Les lignes test, validation, holdout et dev sont exclues. Une entrée absente renvoie `UNRESOLVED`; une correspondance sur support fini ne prouve pas la généralisation.
- Une trace filesystem/process/network passive décrit des événements observés, pas une vérité entrée-sortie d'un algorithme. Elle produit des candidats de méthode (séquence observée) et de règle (profil de schéma) marqués `MANUAL_REVIEW_REQUIRED`. Ils ne deviennent pas des compétences exécutables sans validation et contrat supplémentaires.
- Une compétence de base est un mécanisme réutilisable; une méthode est un enchaînement opératoire; une règle est une contrainte ou condition déclarative. Les composants existants et acquis sont visibles dans le catalogue intégré.

La promotion d'une brique suit au minimum: provenance vérifiée, schéma analysé, périmètre de validité explicite, entrées ambiguës exclues, test de reproduction hors source et cas tenue à l'écart indépendant. Les critères d'acceptation peuvent être déclarés par l'utilisateur; le processus ne doit pas injecter la réponse attendue au modèle durant l'apprentissage.

6 Programmation par composition et compilation

Le programme déclaratif décrit un graphe de nœuds typés, construits avec des ID de compétences présents au catalogue, des méthodes comme `M-SEQUENCE`, et des règles du langage autorisé. Le bouton de compilation écrit un artefact JSON versionné dans `workspace/programs/compiled`. La validation rejette les références inconnues et les opérations arbitraires. L'artefact compilé est ensuite testable et exécutable via le moteur de composition AAM.

Les briques apprises depuis observation sont intégrées au catalogue, mais les candidats passifs gardent un statut explicite non exécutable. Le compilateur ne fait pas comme si un schéma ou une séquence d'événements suffisait à reconstruire une application entière.

7 Modèles locaux et endpoints

Backend	Contrat et limite
`transformers_local`	Dossier complet de modèle/tokenizer/poids déjà présent; chargement local, `trust_remote_code=False`. Convient aux dossiers Transformers.
`pytorch_torchscript`	Charge un module local exporté TorchScript par `torch.jit.load`; le prompt d'inférence est un JSON tensoriel `input`, `inputs` ou `kwargs`. La sortie tensorielle est sérialisée en JSON. CPU par défaut, CUDA seulement si explicitement demandé et disponible; aucun fallback vers un pickle arbitraire.
`pytorch_adapter`	Adaptateur Python local choisi explicitement; contrat `load_model(context)` et `generate(model,prompt,options)`. Un state_dict brut ne décrit ni l'architecture ni le tokenizer; l'adaptateur doit déclarer l'architecture du propriétaire.
`ollama`	Appel HTTP au serveur Ollama déjà installé et démarré; l'URL et le modèle sont renseignés; test explicite pour confirmer que le serveur répond.
`openai_compatible`	Endpoint explicitement indiqué; les secrets sont lus depuis une variable d'environnement et ne sont pas conservés dans la configuration en clair.
`aam workspace` `peer`	Référence vers un workspace AAM local existant; ne transforme pas magiquement un ZIP en modèle exécutable.

Les politiques diffèrent: inférence seule, export de paires supervisées, entraînement LoRA pour Transformers compatibles ou hook d'entraînement d'un adaptateur spécifique. La présence d'une politique dans le registre n'est pas la preuve que les poids ont été entraînés; la sortie et l'évaluation doivent être vérifiées séparément. Les dépendances optionnelles restent explicites.

8 Ergonomie

Les onglets Observer, RAG, Sondes, Compétences, Autonomie, Entraînement, Internet/API, Modèles locaux, Programmation et Réglages utilisent un conteneur canvas avec barre verticale et gestion du défilement à la molette. Les éditeurs de texte structurés conservent leur propre barre. Des sélecteurs permettent de choisir un PID depuis les processus visibles et d'énumérer les interfaces si TShark est installé.

9 Préservation du RAG historique

Le fichier `aam_o93/semantic_rag.py`, le retrieval O92 et les tests RAG historiques listés dans `rapport_O94_3/rag_parity_O94_0_O94_1_O94_2.json` ont des empreintes SHA-256 identiques dans les archives Dmodel O94_0, O94_1 et O94_2. O94_3 les conserve sans changement; les tests de RAG seront rejoués et le manifeste final prouvera leur présence. Cette vérification de contenu n'est pas à elle seule une preuve de qualité de retrieval, qui est évaluée par les tests et jeux dédiés.

10 Commandes d'exécution

```
# Windows: extraire entièrement l'archive, puis lancer
run_AAM_O94_3.bat

# Linux/macOS
```

```
./run_AAM_O94_3.sh

# Préflight et interface
python -m aam_o93 preflight
python -m aam_o93 ui

# Observer passivement un dossier, trace créée par AAM
python -m aam_o93 observe-live --source-kind filesystem \\
  --root ./dossier_que_vous_etes_autorise_a_observer \\
  --authorize-source --seconds 30 --interval 0.25 --max-events 2000

# Observer le PID de ce Python si psutil est disponible
python -m aam_o93 observe-live --source-kind process --pid 1234 \\
  --authorize-source --seconds 15 --interval 0.5

# Lister les compétences et briques disponibles
python -m aam_o93 base-blocks
```

Pour le réseau, installez TShark, choisissez une interface et ajoutez `-source-kind network -interface <id> -authorize-source`. Les droits de capture restent nécessaires.

11 Protocole de validation et interprétation

Les tests unitaires vérifient la génération automatique de JSONL, la modification de fichiers en cours de session, les événements PID lorsque psutil est disponible, l'exigence d'autorisation, l'intégrité de la chaîne de hashes, le rapport final et l'apprentissage excluant les holdouts. Les tests existants d'interface, de modèles, de composition, de RAG et de corpus sont rejoués par lots. Chaque exécution enregistre commande, code de retour, sorties, résultat et éventuel incident. Un test non exécuté ou interrompu n'est jamais compté comme réussi.

12 Conséquences

Théoriques. O94_3 explicite la distinction entre observation passive de sources OS visibles, sondage actif d'une interface connue, import de datasets et inférence d'une compétence. Il rend aussi distincts le support exact d'apprentissage et la généralisation.

Conceptuelles. Une composition reste un graphe déclaratif de briques typées. Une observation passive peut révéler un schéma de comportement; elle ne détermine pas automatiquement toutes les règles internes ou l'algorithme latent d'un logiciel.

Opérationnelles. L'utilisateur peut faire démarrer une collecte sans créer le JSONL à la main, sélectionner un périmètre, observer budgets et erreurs, consulter la trace auto-produite, apprendre des candidats et compiler une composition. Certains niveaux demandent des dépendances système explicites.

13 État et limites de preuve

La livraison déclarera chaque élément selon ses preuves. L'observation filesystem et PID est bornée par ce que le système d'exploitation expose; l'observation réseau demande TShark et les droits système. Aucune garantie de capture de la mémoire d'un processus arbitraire, d'un déchiffrement TLS, de reproduction exacte de tout logiciel ou de généralisation universelle n'est affirmée. Le RAG est conservé et ses tests doivent rester exécutables.